Memoria práctica 3

Seguridad de la información

Luis Martín Pérez – curso 2025/2026

Índice

1 Generación de claves asimétricas, firmado de resúmenes y derivación de secretos con claves DH y ECDH....3
Generar un par de claves asimétricas RSA de 2048 bits en formato PEM con contraseña....3
Exportar la clave pública a otro fichero PEM....3
Exportar la clave privada a formato DER y de vuelta a PEM, con otro nombre de fichero. Comprobar que la conversión a formato DER desprotege la clave privada eliminando la contraseña. 3
Con el par de claves asimétricas creadas, firmar y comprobar la firma del resumen con SHA-256 de uno de los ficheros de texto de la práctica anterior....4
Repetir los cuatro pasos anteriores con claves DSA....5
Generar dos pares de claves DH estándar y demostrar que la combinación pública1-privada2 genera el mismo secreto que la combinación privada1-pública2....7
Generar dos pares de claves DH con curva elíptica X25519 y demostrar que la combinación pública1-privada2 genera el mismo secreto que la combinación privada1-pública2....8
Comparar los tamaños de los dos secretos generados anteriormente....10
2 Intercambio de información segura con cifrado asimétrico de clave simétrica y firma....10
Generar una pareja de claves RSA de 2048 bits en ficheros anapub.pem y anapriv.pem y otra pareja de claves del mismo tipo bertopub.pem y bertopriv.pem. Proteger ambas claves privadas con contraseña “*anak*” y “*bertok*” respectivamente....10
2.A Ana y Berto han intercambiado claves públicas, A continuación realizaremos el trabajo de Ana.…12
Cifrar un fichero de texto con AES-256 en modo CBC, con clave y vector escogidos por el estudiante y sin sal, con salida en formato BASE64 a un fichero llamado cifrado.txt....13
Crear un fichero de texto hexadecimal con la concatenación de la clave y el vector utilizados, de nombre claves.hex y cifrarlo con la clave pública bertopub.pem, con salida en formato binario a un fichero claves.bin....14
Convertir claves.bin a claves.txt en formato BASE64....15
Obtener un resumen sha256 en binario del fichero de texto original con nombre resumen.bin y firmarlo, con salida en formato BASE64 a un fichero llamado firma.txt....16
2.B En este momento, Ana enviaría los tres ficheros obtenidos cifrado.txt, claves.txt y firma.txt junto a la meta-información relativa a los algoritmos utilizados a Berto....16
Convertir el fichero claves.txt a fichero binario claves2.bin. Descifrar el fichero claves2.bin con la clave privada bertopriv.pem con salida a un fichero claves2.hex....17
Extraer de claves2.hex la clave y el vector en formato hexadecimal. Descifrar el fichero cifrado.txt con la clave y el vector obtenidos de claves2.hex y salida al fichero mensaje2.txt....18
Verificar que el fichero firma.txt, convertido a binario y descifrado con la clave pública de Ana, coincide con un resumen sha256 del fichero mensaje2.txt....18
Bibliografía....20

1 Generación de claves asimétricas, firmado de resúmenes y derivación de secretos con claves DH y ECDH

Generar un par de claves asimétricas RSA de 2048 bits en formato PEM con contraseña

Para cualquier proceso de generación de claves se hace uso de la utilidad `openssl genpkey`, ver figura 1.a.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3]
└─$ openssl genpkey -out rsa-2048-pkey.pem -algorithm RSA -aes-256-cbc
.......+.....+.+..+.......+......+..+....+......+...+..+...+......++++++++++++++++++++++++++
+++++++++++++++*.+.....+...+.......+..+.......+...+..+......+............+.......+...+.....
.......................+.......+...+.........+.....+.+.........+......+.....+.......+.......
....+......+...++++++
.+++++++++++++++++++++++++++++++++++++*......+...+..+...+......+.+.......+...+.....+.+....
........++++++++++++++++++++++++++++++++++++++*..+.+.....+.+..............+....+..........
.+......+.......+........+.........+......+.+........+.......+.....+................+......
+......+.........+.+...+.........+.....+...+....+......+...+.....+................+......+.
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
```

Figura 1.a – Generación de clave RSA protegida con contraseña.

Exportar la clave pública a otro fichero PEM

Con la utilidad `openssl pkey` se pueden realizar operaciones sobre las claves, en este caso se extrae la clave pública de la siguiente manera (véase figura 1.b).

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3]
└─$ openssl pkey -in rsa-2048-pkey.pem -inform PEM -pubout -out rsa-2048-pub.pem
Enter pass phrase for rsa-2048-pkey.pem:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3]
└─$ ls -lh
total 8.0K
-rw------- 1 kali kali 1.9K Feb 19 18:56 rsa-2048-pkey.pem
-rw-rw-r-- 1 kali kali  451 Feb 19 19:04 rsa-2048-pub.pem
```

Figura 1.b – Extracción de la clave pública dada la clave privada.

Exportar la clave privada a formato DER y de vuelta a PEM, con otro nombre de fichero. Comprobar que la conversión a formato DER desprotege la clave privada eliminando la contraseña.

Como primer paso, se genera un nuevo fichero en formato DER. Notése como se requiere la contraseña en la figura 1.c.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3]
└─$ openssl pkey -in rsa-2048-pkey.pem -inform PEM -outform DER -out rsa-2048-pkey.der
Enter pass phrase for rsa-2048-pkey.pem:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3]
└─$ ls -l
total 12
-rw------- 1 kali kali 1190 Feb 19 19:07 rsa-2048-pkey.der
-rw------- 1 kali kali 1886 Feb 19 18:56 rsa-2048-pkey.pem
-rw-rw-r-- 1 kali kali  451 Feb 19 19:04 rsa-2048-pub.pem
```

Figura 1.c – Conversión de la clave de formato PEM a DER.

Se crea otro fichero PEM a partir de la conversión de la contraseña en formato DER. Véase en la figura 1.d que ya no se requiere contraseña, confirmando que el formato DER carece de protección con contraseña. También es importante indicar que el fichero `rsa-2048-pkey-paswordless.pem` ha perdido información respecto al original, `rsa-2048-pkey.pem`, lo que quiere decir que la contraseña se ha perdido.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3]
└─$ openssl pkey -in rsa-2048-pkey.der -inform DER -outform PEM -out rsa-2040-pkey-paswordless.pem

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3]
└─$ ls -lh
total 16K
-rw------- 1 kali kali 1.7K Feb 19 19:10 rsa-2040-pkey-paswordless.pem
-rw------- 1 kali kali 1.2K Feb 19 19:07 rsa-2048-pkey.der
-rw------- 1 kali kali 1.9K Feb 19 18:56 rsa-2048-pkey.pem
-rw-rw-r-- 1 kali kali  451 Feb 19 19:04 rsa-2048-pub.pem
```

Figura 1.d – El nuevo fichero PEM ha perdido la contraseña, al igual que el formato DER.

Con el par de claves asimétricas creadas, firmar y comprobar la firma del resumen con SHA-256 de uno de los ficheros de texto de la práctica anterior.

Primero se firma el resumen con la utilidad `openssl pkeyutl` y la opción `-sign`, es importante entender que la firma requiere el uso de la clave privada. (Ver figura 1.e)

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3]
└─$ openssl pkeyutl -sign -in ../practica-2/text.sha256 -out text-signature -inkey rsa-2048-pkey.pem
Enter pass phrase for rsa-2048-pkey.pem:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3]
└─$ ls -lh
total 20K
-rw------- 1 kali kali 1.7K Feb 19 19:10 rsa-2040-pkey-paswordless.pem
-rw------- 1 kali kali 1.2K Feb 19 19:07 rsa-2048-pkey.der
-rw------- 1 kali kali 1.9K Feb 19 18:56 rsa-2048-pkey.pem
-rw-rw-r-- 1 kali kali  451 Feb 19 19:04 rsa-2048-pub.pem
-rw-rw-r-- 1 kali kali  256 Feb 19 19:34 text-signature
```

Figura 1.e – Firma de un resumen mediante la clave generada anteriormente.

El siguiente paso es muy sencillo, el agente que quiera comprobar la firma, tendría que usar la clave pública y el resumen del fichero con la misma utilidad, pero esta vez se usa la opción `-verify`.(Ver figura 1.f)

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3]
└─$ openssl pkeyutl -verify -in ../practica-2/text.sha256 -pubin -inkey rsa-2048-pub.pem -sigfile text-signature
Signature Verified Successfully
```

Figura 1.f – Verificación exitosa de la firma.

Repetir los cuatro pasos anteriores con claves DSA

A diferencia de RSA, openssl no tiene un comando que genere claves DSA en un solo paso. Primero, se requiere generar los parámetros para generar diferentes claves, estos parámetros son los números p, q y g[1]. (Ver figura 1.g)

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl genpkey -genparam -algorithm DSA -out dsa-2048-pa
rams.pem -pkeyopt dsa_paramgen_bits:2048
.......+..............+..+.......++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++++*
.+.....+..+........+..+..................+............+......
.+....+.....+.+.....+.............+...+..+....+...+.....+....
................+.........................+................
.+.....+.....+.+..............+...+.......+..........+....+.+
.....+........+..............+...........+...+..........+...
...+.......+........+.......+..+...+........+...............
............+....+....+..+...+....+............+...+........
+.+.......+..........+..................+..+...+....+....+...
..+............+.......+++++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++*
```

Figura 1.g – Cálculo de parámetros DSA para generación de par de claves.

Una vez se tienen los parámetros, se puede generar el par de claves. En el caso de DSA, la clave privada consiste en la selección aleatoria de un número dentro del conjunto $\{1,\ldots,q-1\}$. (Ver figura 1.h)

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl genpkey -paramfile dsa-2048-params.pem -aes256 -o
ut dsa-2048-pkey.pem
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
```

Figura 1.h – Generación de clave privada a partir de los parámetros DSA.

La clave pública es el resultado de la operación $g^x \, mod \, p$, donde x es la clave privada. En la figura 1.i, se puede observar como el comando es el mismo que con la clave RSA.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl pkey -in dsa-2048-pkey.pem -inform PEM -pubout -out
dsa-2048-pub.pem
Enter pass phrase for dsa-2048-pkey.pem:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ ls -lh dsa*
-rw-rw-r-- 1 kali kali  816 Feb 27 16:49 dsa-2048-params.pem
-rw------- 1 kali kali 1.1K Feb 27 16:53 dsa-2048-pkey.pem
-rw-rw-r-- 1 kali kali 1.2K Feb 27 17:01 dsa-2048-pub.pem
```

Figura 1.i – Generación de la clave pública a partir de la privada en DSA.

Ya que se tiene el par de claves, el siguiente y último paso es realizar la operación de firma y su verificación. Estos pasos carecen de alguna diferencia con el proceso realizado con RSA. En la figura 1.j se realiza la firma, y en la figura 1.k se verifica de forma existosa.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl pkeyutl -sign -in ../../practica-2/ej1/text.sha256 -
out text-signature -inkey dsa-2048-pkey.pem
Enter pass phrase for dsa-2048-pkey.pem:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ ls -lh dsa* text-signature
-rw-rw-r-- 1 kali kali  816 Feb 27 16:49 dsa-2048-params.pem
-rw------- 1 kali kali  850 Feb 27 17:02 dsa-2048-pkey.der
-rw------- 1 kali kali  879 Feb 27 17:03 dsa-2048-pkey-paswordle
ss.pem
-rw------- 1 kali kali 1.1K Feb 27 16:53 dsa-2048-pkey.pem
-rw-rw-r-- 1 kali kali 1.2K Feb 27 17:01 dsa-2048-pub.pem
-rw-rw-r-- 1 kali kali   63 Feb 27 17:06 text-signature
```

Figura 1.j – Firma con la clave pública DSA.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl pkeyutl -verify -in ../../practica-2/ej1/text.sha256
 -pubin -inkey dsa-2048-pub.pem -sigfile text-signature
Signature Verified Successfully
```

Figura 1.k – Verificación exitosa de la firma DSA.

Generar dos pares de claves DH estándar y demostrar que la combinación pública1-privada2 genera el mismo secreto que la combinación privada1-pública2

El proceso de generación de claves a partir de DH es muy parecido a DSA, primero se calculan los parámetros necesarios.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl genpkey -genparam -algorithm DH -out dh-params.pem
.....................................................................
.....................................................................
.............................+.......................................
```

Figura 1.l – Generación de parámetros DH para la generación de claves.

De la misma manera que DSA, los parámetros se pueden reutilizar para generar varios pares de claves.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl genpkey -paramfile dh-params.pem -out privada1.pem -
aes256
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl genpkey -paramfile dh-params.pem -out privada2.pem -
aes256
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl pkey -in privada1.pem -inform PEM -pubout -out publi
ca1.pem
Enter pass phrase for privada1.pem:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl pkey -in privada2.pem -inform PEM -pubout -out publi
ca2.pem
Enter pass phrase for privada2.pem:
```

Figura 1.m – Creación de dos pares de claves a partir de los parámetros DH.

A continuación, se deriva el secreto compartido con la opción `-derive` del comando `openssl pkeyutl`. La secuencia utilizada es clave privada 1, clave pública 2.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl pkeyutl -inkey privada1.pem -keyform PEM -derive -pe
erform PEM -peerkey publica2.pem -out secreto-priv1-pub2
Enter pass phrase for privada1.pem:
```

Figura 1.n – Derivación del secreto compartido priv1-pub2.

Se genera otro secreto compartido con la combinación complementaria y se verifica que se trata del mismo contenido.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl pkeyutl -inkey privada2.pem -keyform PEM -derive -pe
erform PEM -peerkey publica1.pem -out secreto-pub1-priv2
Enter pass phrase for privada2.pem:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ ls -lh *{1,2}.pem secreto*
-rw------- 1 kali kali 976 Feb 27 17:23 privada1.pem
-rw------- 1 kali kali 976 Feb 27 17:23 privada2.pem
-rw-rw-r-- 1 kali kali 800 Feb 27 17:24 publica1.pem
-rw-rw-r-- 1 kali kali 800 Feb 27 17:24 publica2.pem
-rw-rw-r-- 1 kali kali 256 Feb 27 17:29 secreto-priv1-pub2
-rw-rw-r-- 1 kali kali 256 Feb 27 17:31 secreto-pub1-priv2

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ diff <(xxd secreto-priv1-pub2) <(xxd secreto-pub1-priv2)

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
```

Figura 1.o – Comprobación de que se genera el mismo secreto compartido.

Generar dos pares de claves DH con curva elíptica X25519 y demostrar que la combinación pública1-privada2 genera el mismo secreto que la combinación privada1-pública2.

Esta vez las claves se generan con una sola línea. Parece ser que los algoritmos criptográficos más usados y probados son lo que `openssl` hace más fáciles de usar.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl genpkey -algorithm X25519 -out privada1.pem -aes256
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl genpkey -algorithm X25519 -out privada2.pem -aes256
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl pkey -in privada1.pem -inform PEM -pubout -out publi
ca1.pem
Enter pass phrase for privada1.pem:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl pkey -in privada2.pem -inform PEM -pubout -out publi
ca2.pem
Enter pass phrase for privada2.pem:
```

Figura 1.p – Generación de dos pares de claves X25519.

Se derivan los secretos compartidos mediante combinaciones diferentes pública-privada y se verifica que es el mismo contenido. (Ver figura 1.r)

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl pkeyutl -inkey privada1.pem -keyform PEM -derive -pe
erform PEM -peerkey publica2.pem -out secretoX25519-priv1-pub2
Enter pass phrase for privada1.pem:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ openssl pkeyutl -inkey privada2.pem -keyform PEM -derive -pe
erform PEM -peerkey publica1.pem -out secretoX25519-pub1-priv2
Enter pass phrase for privada2.pem:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ diff <(xxd secretoX25519-priv1-pub2) <(xxd secretoX25519-pub
1-priv2)

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
```

Figura 1.r – Comprobación de que se deriva el mismo secreto con X25519.

Comparar los tamaños de los dos secretos generados anteriormente.

El secreto compartido de DH estándar ocupa ocho veces el espacio del secreto compartido de ECDH-X25519. Además, las operaciones aritméticas en grupos multiplicativos de gran tamaño hacen que DH sea mucho más lento que X25519.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej1]
└─$ ls -lh secreto*
-rw-rw-r-- 1 kali kali 256 Feb 27 17:29 secreto-priv1-pub2
-rw-rw-r-- 1 kali kali 256 Feb 27 17:31 secreto-pub1-priv2
-rw-rw-r-- 1 kali kali  32 Feb 27 17:47 secretoX25519-priv1-pub2
-rw-rw-r-- 1 kali kali  32 Feb 27 17:47 secretoX25519-pub1-priv2
```

Figura 1.s – Comparación de los tamaños de los secretos entre DH y X25519.

2 Intercambio de información segura con cifrado asimétrico de clave simétrica y firma

Generar una pareja de claves RSA de 2048 bits en ficheros `anapub.pem` y `anapriv.pem` y otra pareja de claves del mismo tipo `bertopub.pem` y `bertopriv.pem`. Proteger ambas claves privadas con contraseña *"anak"* y *"bertok"* respectivamente

En la figura 2.a, se puede ver como se generan dos claves privadas para Ana y Berto respectivamente. Se escriben las contraseñas igual que en el enunciado.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2]
└─$ openssl genpkey -algorithm RSA -aes256 -out anapriv.pem
.+.......+....+.....+..............+ ... + ... + ... +....+.....+ ...
.......+ .. +.+.......+......+....+......+ ... +......+.....+.+ ... +
.....+++++++++++++++++++++++++++++++++++++++*........+......+.+.
.. + ... + .. + ... +.......+ ... + .. +......+.......+........+.+++++++++
++++++++++++++++++++++++++++* ... +....+ ... + .. +........+......+.
............+..............+ .. +......+ ... +.+ ... +......+ .. + ... +.
... ++++++
......+ .. +.+ ... + .. +++++++++++++++++++++++++++++++++++++++*......
+......+.+......+.....+ ... +......+........+....+......+......+.
.. + .. + ... +.....................+.+......+ ... +......+++++++++++++
++++++++++++++++++++++++++*.............+ ... +........+ ... +.....
.. + ... +........+........+....+....................+.......+ .. +.
.. + ... +......+.........+........+ .. + ... ++++++
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2]
└─$ openssl genpkey -algorithm RSA -aes256 -out bertopriv.pem
......+.....+.....+.+ .. +.+................+++++++++++++++++++
+++++++++++++++++++*....+++++++++++++++++++++++++++++++++++++++*
....+ ... +......+............+........+ ... +.+......+ .. +.+ .. +....
....................+.+........+.+......+.....+....+............
.. +.................+ .. + ... +.......+......+........+....+.....
.+...........+.+ .. +.+.....+...........+.........+.+............
.....+............+ ... +......+.........++++++
.. +......+ .. +....+......+ ... + .. +....+ ... +++++++++++++++++++++++
+++++++++++++++*...........+ ... +.+......+++++++++++++++++++++++++
+++++++++++++++*........+ ... +................+.....+.+..........
.....+........+.+...............+..................+ ... + ... +++++
+
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
```

Figura 2.a – Generación de claves privadas para Ana y Berto.

A partir de las claves privadas, se generan las claves públicas correspondientes. (Ver figura 2.b)

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2]
└─$ openssl pkey -in anapriv.pem -pubout -out anapub.pem
Enter pass phrase for anapriv.pem:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2]
└─$ openssl pkey -in bertopriv.pem -pubout -out bertopub.pem
Enter pass phrase for bertopriv.pem:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2]
└─$ ls -lh
total 16K
-rw------- 1 kali kali 1.9K Feb 27 18:52 anapriv.pem
-rw-rw-r-- 1 kali kali  451 Feb 27 18:54 anapub.pem
-rw------- 1 kali kali 1.9K Feb 27 18:52 bertopriv.pem
-rw-rw-r-- 1 kali kali  451 Feb 27 18:54 bertopub.pem
```

Figura 2.b – Par de claves completo para Ana y Berto.

2.A Ana y Berto han intercambiado claves públicas, A continuación realizaremos el trabajo de Ana.

Se crean dos directorios, `ana` y `berto` para distinguir los roles de cada agente y se simula el intercambio de claves públicas mediante la copia. (Ver figura 2.A.a)

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2]
└─$ mkdir berto ana

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2]
└─$ cp anapub.pem berto

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2]
└─$ cp bertopub.pem ana

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2]
└─$ mv berto*.pem berto

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2]
└─$ mv ana*.pem ana

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2]
└─$ ls -l
total 8
drwxrwxr-x 2 kali kali 4096 Feb 27 18:56 ana
drwxrwxr-x 2 kali kali 4096 Feb 27 18:56 berto

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2]
└─$ tree .
.
├── ana
│   ├── anapriv.pem
│   ├── anapub.pem
│   └── bertopub.pem
└── berto
    ├── anapub.pem
    ├── bertopriv.pem
    └── bertopub.pem

3 directories, 6 files
```

Figura 2.A.a – Intercambio de claves entre Ana y Berto.

Cifrar un fichero de texto con AES-256 en modo CBC, con clave y vector escogidos por el estudiante y sin sal, con salida en formato BASE64 a un fichero llamado cifrado.txt.

Se escribe un contenido sumamente confidencial en el fichero `texto.txt` y se cifra según los requisitos establecidos. (Ver figuras 2.A.b y 2.A.c)

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ cat texto.txt | fold -w 41
Hola Berto, el atentado es mañana a las
15:37 en 28.100382141078484, -15.45671247
6611237
```

Figura 2.A.b – Contenido confidencial del fichero.

Debido a que se debe escoger una clave e iv propios, se opta por escribir dos mensajes de texto de 16 Bytes y 32 Bytes, coincidiendo con los tamaños de la clave e iv. Luego se convierten a su representación en hexadecimal con el comando `xxd -p` y se añaden como parámetros en el cifrado.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ echo -n "Tesoro y meteoro" | wc -c
16

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ echo -n "Tesoro y meteoro" | xxd -p
5465736f726f2079206d6574656f726f

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ echo -n "Tesoros y meteoros son valiosos." | wc -c
32

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ echo -n "Tesoros y meteoros son valiosos." | xxd -p
5465736f726f732079206d6574656f726f7320736f6e2076616c696f736f
732e

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ openssl enc -aes-256-cbc -a -in texto.txt -K 5465736f726f732
079206d6574656f726f7320736f6e2076616c696f736f732e -iv 5465736f72
6f2079206d6574656f726f -nosalt -out cifrado.txt
```

Figura 2.A.c – Cifrado del mensaje con salida en Base64 y sin sal.

Crear un fichero de texto hexadecimal con la concatenación de la clave y el vector utilizados, de nombre claves.hex y cifrarlo con la clave pública bertopub.pem, con salida en formato binario a un fichero claves.bin

Primero se escriben la clave y el vector en el fichero claves.hex. La concatenación se hace en líneas diferentes, es decir, Berto sabrá que en la línea 1 se encuentra la clave (debido al tamaño más largo) y que en la línea dos está el vector. (También se podría haber llegado previamente a un consenso para elegir la clave y vector y quitaría la necesidad de compartir el fichero en la figura 2.A.e)

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ echo -n "Tesoros y meteoros son valiosos." | xxd -p > claves
.hex

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ echo -n "Tesoro y meteoro" | xxd -p >> claves.hex

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ cat claves.hex
5465736f726f732079206d6574656f726f7320736f6e2076616c696f736f
732e
5465736f726f2079206d6574656f726f
```

Figura 2.A.d – Concatenación de la clave y vector.

Se cifran las claves con la clave pública para que Berto sea la única persona que pueda descifrarlo con su clave privada.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ openssl pkeyutl -encrypt -in claves.hex -pubin -inkey bertop
ub.pem -out claves.bin
```

Figura 2.A.e – Cifrado de las claves en un fichero binario.

Convertir claves.bin a claves.txt en formato BASE64

Se codifica a Base64 con el comando openssl enc y la opción -a. En este momento, el fichero `claves.hex` se debería eliminar.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ openssl enc -a -in claves.bin -out claves.txt

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ ls -lh
total 32K
-rw------- 1 kali kali 1.9K Feb 27 18:52 anapriv.pem
-rw-rw-r-- 1 kali kali  451 Feb 27 18:54 anapub.pem
-rw-rw-r-- 1 kali kali  451 Feb 27 18:55 bertopub.pem
-rw-rw-r-- 1 kali kali  130 Feb 27 19:35 cifrado.txt
-rw-rw-r-- 1 kali kali  256 Feb 27 19:45 claves.bin
-rw-rw-r-- 1 kali kali   99 Feb 27 19:39 claves.hex
-rw-rw-r-- 1 kali kali  350 Feb 27 19:46 claves.txt
-rw-rw-r-- 1 kali kali   90 Feb 27 19:05 texto.txt
```

Figura 2.A.f – Codificación de claves.bin en Base64.

Obtener un resumen sha256 en binario del fichero de texto original con nombre resumen.bin y firmarlo, con salida en formato BASE64 a un fichero llamado firma.txt.

Como ya se ha visto anteriormente, se genera el resumen en formato binario, se firma el resumen con la clave privada de Ana y se codifica en base64. En este momento, el fichero `texto.txt` se debería eliminar. (Ver figura 2.A.g)

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ openssl dgst -sha256 -binary texto.txt > resumen.bin

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ openssl pkeyutl -sign -in resumen.bin -inkey anapriv.pem | o
penssl enc -a -out firma.txt
Enter pass phrase for anapriv.pem:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ cat firma.txt
0KHRPsEXLkjPMH/vCBQzeHx7JIjsKVbU7JNdsh/AK9+jscLH61CyJtKO8XwsGyKJ
X6n3Dk2hj5sgMkr2BFld3Zwxq5Ye3vy5WoiW2hAeYe0zS+Gt9cEi+V1tsu9GLabX
RE/jGI9JC1mxf/c03HXdLczsX08ObRMB77OfMPQlWZ9PNwctH0Y0IL3Kad9ffKju
72QhyYHsj02jcQM/1UDpxpxcI5DWksGzfOuK83Qi39VX1nYUgBxFmRYprizdzKWL
c4AlrqqQgKikfmSr8+U6Edelk6/XOcfIQVedZujvUKBGEfVmtW1xVsChq415us8j
ToAVsViZ++JaAIHJY5sSog==
```

Figura 2.A.g – Generación de ficheros para verificar integridad e identidad del mensaje texto.txt.

2.B En este momento, Ana enviaría los tres ficheros obtenidos cifrado.txt, claves.txt y firma.txt junto a la meta-información relativa a los algoritmos utilizados a Berto.

Se simula el envío mediante la copia de un directorio a otro en la figura 2.B.a.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ cp cifrado.txt claves.txt firma.txt ../berto

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/ana]
└─$ cd ../berto

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/berto]
└─$ ls -lh
total 24K
-rw-rw-r-- 1 kali kali  451 Feb 27 18:55 anapub.pem
-rw------- 1 kali kali 1.9K Feb 27 18:52 bertopriv.pem
-rw-rw-r-- 1 kali kali  451 Feb 27 18:54 bertopub.pem
-rw-rw-r-- 1 kali kali  130 Feb 27 20:01 cifrado.txt
-rw-rw-r-- 1 kali kali  350 Feb 27 20:01 claves.txt
-rw-rw-r-- 1 kali kali  350 Feb 27 20:01 firma.txt
```

Figura 2.B.a – Envío de información a Berto.

Convertir el fichero claves.txt a fichero binario claves2.bin. Descifrar el fichero claves2.bin con la clave privada bertopriv.pem con salida a un fichero claves2.hex

Se hace el proceso inverso a Ana, es decir, añadiendo la opción -d o –decrypt en cada comando, primero se decodifica de base64 a binario y luego se descifra con la clave privada de Berto para obtener el fichero original de texto plano. (Ver figura 2.B.b)

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/berto]
└─$ openssl enc -a -d -in claves.txt -out claves.bin

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/berto]
└─$ openssl pkeyutl -decrypt -in claves.bin -inkey bertopriv.pem
 -out claves.hex
Enter pass phrase for bertopriv.pem:

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/berto]
└─$ cat claves.hex
5465736f726f732079206d6574656f726f7320736f6e2076616c696f736f
732e
5465736f726f2079206d6574656f726f
```

Figura 2.B.b – Extracción de las claves de cifrado.

Extraer de claves2.hex la clave y el vector en formato hexadecimal. Descifrar el fichero cifrado.txt con la clave y el vector obtenidos de claves2.hex y salida al fichero mensaje2.txt

Se hace uso del cifrado simétrico AES-256-CBC, junto a las claves extraídas del fichero y siguiendo el convenio establecido. Se revela el mensaje confidencial que ahora solo conocen Ana y Berto. (Ver figura 2.B.c)

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/berto]
└─$ cat claves.hex
5465736f726f732079206d6574656f726f7320736f6e2076616c696f736f7
32e
5465736f726f2079206d6574656f726f

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/berto]
└─$ openssl enc -a -d -in cifrado.txt -out mensaje2.txt -iv 5
465736f726f2079206d6574656f726f -K 5465736f726f732079206d6574
656f726f7320736f6e2076616c696f736f732e -aes-256-cbc

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/berto]
└─$ cat mensaje2.txt
Hola Berto, el atentado es mañana a las 15:37 en 28.100382141
078484, -15.456712476611237
```

Figura 2.B.c – Descifrado del texto cifrado para la obtención del mensaje original.

Verificar que el fichero firma.txt, convertido a binario y descifrado con la clave pública de Ana, coincide con un resumen sha256 del fichero mensaje2.txt

Se hacen las comprobaciones de integridad e identidad para verificar que el mensaje no ha sido modificado en su envío y que, en efecto proviene de Ana. (Se asume que el intercambio de claves se hice mediante un medio completamente P2P)

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/berto]
└─$ openssl enc -a -d -in firma.txt -out firma.bin

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/berto]
└─$ cat firma.bin
C◆>◆.H◆03x|{$◆◆)V◆◆◆]◆◆+l◆◆◆◆P◆&R◆|,◆_◆◆M◆◆◆ 2J◆Y]1◆◆◆◆◆Z◆◆◆
a◆3K◆◆◆◆"◆]m◆◆F-◆◆DO◆ ◆I
                        Y◆◆4◆u◆-◆◆_Om0◆%Y◆O7-F4 ◆◆i◆_|◆◆◆d!?
◆◆M◆q?◆@◆W\#◆◆◆|◆◆◆t"◆◆W◆v◆E◆)◆,◆◆s◆%◆◆◆◆◆◆~d◆◆◆:9◆◆◆◆◆AW◆f◆
◆P◆F◆f◆mqV◆◆◆◆y◆◆#N◆◆X◆◆◆Z◆◆c◆◆
```

Figura 2.B.d – Decodificación de la firma.

```
┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/berto]
└─$ openssl pkeyutl -verifyrecover -in firma.bin -pubin -inke
y anapub.pem > resumen-ana.sha256

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/berto]
└─$ openssl dgst -sha256 -binary mensaje2.txt | diff - resume
n-ana.sha256

┌──(luis-martin-perez@cyberbox)-[~/SI/practica-3/ej2/berto]
```

Figura 2.B.e – Verificación de la firma y resumen del texto.

Bibliografía

[1] *Wikipedia contributors, "Digital Signature Algorithm – 1. Key generation," Wikipedia, The Free Encyclopedia. [Online]. Available: https://en.wikipedia.org/wiki/Digital_Signature_Algorithm#1._Key_generation. [Accessed: Feb. 28, 2026].*

[2] *OpenSSL Software Foundation, "openssl(1) — OpenSSL command line tool," OpenSSL Documentation. [Online]. Available: https://docs.openssl.org/master/man1/. [Accessed: Mar. 2, 2026].*